CEGAR-Harness: Regression-Aware Context Compaction and Versioned Witness Replay for Autonomous Coding Agents

Iyed Mohamed
ESPRIT — School of Engineering and Technology, Tunis, Tunisia
`iyed.mohamed@esprit.tn` | Open-Source Repository: `https://github.com/iyedM/cegar-harness`
Technical Report — October 2026 (Version 0.2.0)

Abstract—Long-horizon autonomous coding agents (such as SWE-agent, OpenHands, and Claude Code) accumulate large tool observation traces from test runners, static analyzers, and file inspections. When the trajectory approaches the context budget or experiences attention degradation, the agent harness invokes a context compaction operator—typically observation masking or narrative LLM summarization. However, standard compaction operators discard exact falsifying inputs and assertion predicates from earlier failed repair attempts. Consequently, in post-compaction continuation turns, agents frequently re-propose previously refuted patches or syntactically distinct rewrites of the same logical defect, a failure mode we formalize as *Post-Compaction Regression Loops (PCRL)*.

In this report, we present **CEGAR-Harness**, an open-source middleware and evaluation harness inspired by Counterexample-Guided Abstraction Refinement (CEGAR). Whenever a candidate patch fails a deterministic verification oracle, CEGAR-Harness extracts a structured failure witness and stores it in a persistent, contract-versioned SQLite ledger (`CounterexampleLedger`). During context compaction, verbose tool logs are pruned while active witness predicates are pinned into the compacted header. Before admitting any post-compaction patch to full test-suite execution, a three-state `RegressionGate` (`PASS`, `REGRESSION_DETECTED`, `INCONCLUSIVE`) evaluates the patch via an optional syntactic AST canonicalization pre-filter and isolated subprocess **Witness Replay**. We evaluate CEGAR-Harness across a 30-task multi-turn Python code-repair benchmark spanning 6 bug categories—including syntactic bug variants, evolving specification contracts (`v1 → v2`), and partial witness coverage challenges requiring multi-round refinement. Empirical results demonstrate that CEGAR-Harness compresses context tokens by **48.26%**, reduces unblocked known regressions from **100.00%** (under observation masking and lossy summarization) and **16.67%** (under un-gated ledger pinning) to **0.00%**, prevents **100% (5/5)** of false blocks on evolved contracts via its `INCONCLUSIVE` state, and achieves a **93.33%** (95% Wilson CI: `[78.7%, 98.2%]`) two-turn task resolution rate with a median gate latency of **18.89 ms**.

1. Introduction & Motivation

Autonomous software engineering agents operate in an iterative closed loop: inspecting repository files, proposing candidate code modifications, executing test suites, and interpreting stack traces. While short single-turn repairs fit comfortably inside modern context windows, realistic debugging sessions spanning 10 to 50 tool invocations rapidly accumulate thousands of tokens of noisy standard output, pytest collection headers, and control-flow dumps. As shown by recent empirical studies on context management (JetBrains Research, 2025; Compact-Bench, October 2026), unconstrained context growth degrades reasoning fidelity ("context rot") and inflates inference cost and latency.

To bound context growth, modern agent harnesses apply **Context Compaction**: compressing or eliding historical turns $1 \dots (t - W)$ while keeping a sliding window of the most recent W turns in full fidelity. Existing harnesses rely primarily on two compaction mechanisms:

1. **Observation Masking (`OBSERVATION_MASKING`):** Replacing older tool observation payloads with placeholder tokens (e.g., `<observation masked>`) while preserving the agent's action history.
2. **Lossy Narrative Summarization (`LOSSY_SUMMARY`):** Prompting an LLM to summarize earlier exploration steps into a concise natural-language paragraph.

Both approaches suffer from a critical structural vulnerability during iterative bug repair: **they discard the exact falsifying witness inputs and assertion failure predicates discovered prior to compaction**. When an agent attempts a plausible-looking patch at Step 2 that fails on a subtle boundary condition (for example, filtering out falsy zero integers `0` when removing `None` values), and the harness subsequently compacts Steps 1–4 at Step 6, the post-compaction context no longer contains the refutation proof. On Step 7, the agent policy frequently re-samples the exact same broken logic—or a syntactically distinct variant such as replacing a `for` loop with a list comprehension—entering a **Post-Compaction Regression Loop (PCRL)**.

1.1 Motivating Concrete Example: Syntactic Bug Re-Introduction After Compaction

Consider Task `T26` from Category 6 (`6_syntactic_variants`) of our benchmark suite: implementing `compact_ints(items: list[int | None]) -> list[int]` to strip `None` elements while preserving all valid integers including `0`.

Table 1: Side-by-side trajectory comparison on Task T26 (`filters.py::compact_ints`) across compaction boundaries.

Trajectory Phase	Standard Harness (`LOSSY_SUMMARY` / `OBSERVATION_MASKING`)	`CEGAR-Harness` (`CEGAR_HARNESS_FULL`)
Step 1–2 (Pre-Compaction Failure)	Agent proposes imperative loop with truthiness check: `if item: out.append(item)`. Witness test `test_zero_is_not_dropped` fails: `AssertionError: [1, 2] != [0, 1, 2]`.	Identical failure occurs. `CEGAR-Harness` records witness `CE-T26-01` in SQLite with `contract_version="v1"`, canonical AST hash `h_AST`, isolated test `test_zero_is_not_dropped`, and failure predicate.

Trajectory Phase	Standard Harness (`LOSSY_SUMMARY` / `OBSERVATION_MASKING`)	`CEGAR-Harness` (`CEGAR_HARNESS_FULL`)
Step 3–6 (Exploration)	Agent inspects downstream call sites and static types (+450 tokens of verbose tool observations).	Identical 450 tokens of repository search and control-flow inspection observations.
Compaction Event	Verbose tool logs are masked or summarized into: *"The agent explored 4 prior steps and encountered 1 test failure."* **The fact that `0` was dropped by truthiness filtering is erased.**	Verbose logs are pruned (**48.26% token reduction**), while `CE-T26-01` is pinned in the header: `Failed test_zero_is_not_dropped: [1, 2] != [0, 1, 2] | AST=9f1c...`
Post-Compaction Turn 1	Agent proposes list comprehension `[x for x in items if x]`. Because no gate exists, it executes against the full test suite and triggers an **unblocked regression**.	Even if the agent proposes the list comprehension variant `[x for x in items if x]` (which has a **different AST hash** than the `for` loop), `RegressionGate` replays `test_zero_is_not_dropped` in `~19 ms`, returns `REGRESSION_DETECTED`, blocks the regression before full suite execution, and converges on `[x for x in items if x is not None]`.

1.2 Research Questions and Contributions

Rather than claiming that AST hashing provides semantic equivalence or that a single witness guarantees total program correctness, this work investigates a focused, testable systems hypothesis: *Can a persistent, contract-versioned counterexample ledger paired with pre-execution isolated witness replay eliminate repeated post-compaction regressions without inducing false blocks when requirements evolve?* We structure our study around four research questions:

- **RQ1 (Compaction vs. Regression & Convergence):** How do five context compaction strategies (`FULL_HISTORY`, `OBSERVATION_MASKING`, `LOSSY_SUMMARY`, `CEGAR_LEDGER_ONLY`, and `CEGAR_HARNESS_FULL`) compare in token compression ratio, witness predicate retention, unblocked regression rate, Turn-1 resolution rate, and two-turn convergence rate when subjected to partial witness coverage?
- **RQ2 (Component Ablation — Syntactic AST vs. Behavioral Replay):** What are the individual contributions of the canonical AST pre-filter and isolated Witness Replay to gate precision, recall, F1-score, and latency, particularly on syntactically distinct bug rewrites (Category 6)?
- **RQ3 (Specification Evolution & False Block Mitigation):** How does unversioned witness replay fail when task specifications evolve across compaction boundaries (Category 5, `v1 → v2`), and how effectively does contract versioning with the `INCONCLUSIVE` gate state eliminate false blocks?
- **RQ4 (Industrial Integration & Overhead):** Can the counterexample ledger and regression gate be exposed as a low-latency middleware via the Model Context Protocol (MCP), atomic Git workspace rollbacks, and LangGraph state interceptors with sub-25 ms median overhead?

2. Background & Related Work

Our design is informed by three converging research threads from 2025–2026: (i) empirical benchmarks of context compaction damage in coding agents, (ii) learned and heuristic context compression algorithms, and (iii) counterexample-guided synthesis and formal repair with LLMs.

Table 2: Positioning of CEGAR-Harness relative to 2025–2026 literature on context compaction and counterexample-guided repair.

System / Study	Primary Scope	Core Mechanism & Findings	Differentiation in `CEGAR-Harness`
Compact-Bench (Cai et al., Oct 2026, `arXiv:2610.31710`)	Benchmarking compaction damage in coding agents	Evaluates post-compaction continuations from identical repository states; demonstrates that mechanical extraction preserves recurring constraints better than free-form LLM summaries.	Complements passive mechanical extraction with an **active 3-state Regression Gate** that executes stored witness tests in an isolated sandbox before admitting candidate patches.
ACON (Kang et al., 2025, `arXiv:2510.00615`)	Adaptive context compression optimization	Optimizes compression rules by contrasting trajectories that succeed before compression against those that fail after compression.	Operates as a **deterministic runtime middleware** requiring zero model training or prompt-optimization loops, backed by SQLite witness persistence.
The Complexity Trap (JetBrains Research, 2025)	Observation masking vs. LLM summarization	Shows that simple observation masking often matches or outperforms expensive LLM trajectory summarization on software engineering benchmarks.	Implements both `OBSERVATION_MASKING` and `LOSSY_SUMMARY` as baselines and shows how augmenting observation pruning with pinned witnesses eliminates post-compaction regressions.
CompactionBench & APEC (Microsoft Research, Oct 2026)	Latency & disruption of compaction in long coding sessions	Quantifies compaction latency spikes and context disruption, proposing asynchronous progressive compaction.	Explicitly instruments and reports median (`18.89 ms`) and p95 (`21.72 ms`) gate latency, decomposing SQLite/AST lookup (`0.36 ms`) from subprocess replay (`11.59 ms`).
ExVerus (ICML 2026), **COVER** & **MENTOR**	Counterexample-guided formal proof & program repair	Uses verifier counterexamples within a single repair loop to guide formal proof synthesis (Verus/Rust) and constraint modeling.	Extends counterexample retention **across context compaction boundaries** and introduces contract-versioned

System / Study	Primary Scope	Core Mechanism & Findings	Differentiation in `CEGAR-Harness`
			`INCONCLUSIVE` gating to handle specification drift.
SWE-bench (Jimenez et al., ICLR 2024 / Verified 2025)	End-to-end GitHub issue resolution	Evaluates full repository patch resolution using Dockerized test suites across real open-source Python projects.	Provides a fast (`< 20s`), self-contained 30-task micro-benchmark with controlled ablation of AST filtering, witness replay, partial witness coverage, and contract evolution.

2.1 Why Passive Context Extraction Alone Is Insufficient

A central insight from *Compact-Bench* (October 2026) is that mechanical extraction of error messages outperforms free-form narrative summarization. However, our ablation between `CEGAR_LEDGER_ONLY` (which pins the exact failure predicate and AST hash into the compacted prompt text without pre-execution gating) and `CEGAR_HARNESS_FULL` reveals two fundamental limitations of prompt-only retention:

1. **Syntactic Isomorphism Blindness:** Even when a prior failure predicate is visible in the prompt, an LLM frequently attempts to fix a bug by rewriting its surface syntax (e.g., converting an imperative loop into a comprehension, or replacing an `if/else` guard with a short-circuit boolean `or` expression) while inadvertently preserving the exact same faulty edge-case behavior. Without executable witness replay, these syntactically distinct regressions pass straight into full test-suite execution.
2. **Partial Witness Coverage & Multi-Constraint Refinement:** A witness extracted from Step 2 only falsifies the specific input vector tested at Step 2. An agent may satisfy that first witness on Post-Compaction Turn 1 while violating a second boundary condition in the specification, requiring iterative multi-round witness accumulation.

3. Formal Problem Formulation & Three-State Regression Gate Semantics

3.1 Agent Trajectory and Compaction Operators

Let an autonomous coding task be defined by a specification S_v at contract version $v \in \{v_1, v_2, \ldots\}$, an initial file mapping $F_0 : Path \to \Sigma^*$, and a reference oracle test suite Φ_{ref}. At discrete step t, the agent trajectory is a sequence of action-observation turns:

$$T_t = \langle S_v, (a_1, o_1), (a_2, o_2), \ldots, (a_t, o_t) \rangle$$

where a_k denotes an agent action (such as proposing a candidate file dictionary F_k or invoking a repository inspection tool) and o_k denotes the resulting tool observation. When the token length $|T_t|_{tok}$ exceeds a compaction threshold, a compaction operator $C(T_t, L_t)$ maps the trajectory and the external counterexample ledger L_t to a compressed context representation $\hat{T}_t$ with sliding window size W:

- **Full History (C_{full}):** $\hat{T}_t = T_t$ (compression ratio = 0.00%).
- **Observation Masking (C_{mask}):** Retains all actions $a_1 \ldots a_t$ and recent observations $o_{t-W+1} \ldots o_t$, but replaces every older tool observation o_k ($k \le t - W$) with an elision marker `<observation masked>`.
- **Lossy Summarization (C_{lossy}):** Replaces the prefix $((a_1, o_1), \ldots, (a_{t-W}, o_{t-W}))$ with an episodic narrative summary s_{narr} that records high-level attempt counts while omitting stack traces, variable bindings, and assertion predicates.
- **CEGAR Compacted (C_{cegar}):** Prunes verbose prefix observations $o_1 \ldots o_{t-W}$ while pinning the active counterexample set $\{c_j \in L_t \mid status(c_j) = ACTIVE\}$ directly beneath S_v.

3.2 Versioned Counterexample Tuple and Canonical AST Pre-Filter

Whenever a candidate patch F_k fails a witness assertion w under contract version v, CEGAR-Harness records a counterexample tuple in the persistent SQLite ledger L:

$$c_k = \langle ce_id, task_id, target_file, target_symbol, v, h_{AST}(F_k), w_{name}, w_{code}, \psi_{fail}, k, status \rangle$$

Here, $h_{AST}(F_k) = SHA256(dump(\alpha(parse(F_k))))$ is computed by parsing each Python source file into its Abstract Syntax Tree, stripping module and function docstrings, and applying a deterministic alpha-renaming transformer (`_AlphaNormalizer`) that maps user-defined local variable and parameter identifiers to canonical positional indices (`_v0, _v1, ...`) in traversal order while preserving Python built-in symbols (`len`, `min`, `max`, `isinstance`, `ValueError`, etc.) and attribute names.

> **Methodological Principle (No False Semantic Equivalence Claims):** Canonical AST hashing is strictly a *syntactic pre-filter* ($O(|F_k|)$ time, median latency `0.30 ms`). It catches exact repeats, formatting/comment edits, and local variable renamings, but **cannot** detect syntactically distinct implementations of the same logical defect. Behavioral detection relies on Stage 2: isolated execution of w_{code}.

3.3 Three-State Regression Gate Decision Automaton

A critical design flaw in naive regression gates is treating every historical test failure as a permanent invariant. In software engineering, requirements frequently evolve mid-session (for example, changing an API from clamping negative values to raising a `ValueError`, or changing HTML escaping to preserve single quotes in `v2`). If a gate blindly enforces a `v1` witness against a valid `v2` patch, it causes a **False Block** (rejecting a correct solution).

To prevent false blocks while maintaining strict regression detection within a contract version, `RegressionGate.evaluate(task_id, F_cand, v_curr)` implements a three-state decision function over *L(task_id)*:

```
Input: task_id, candidate files F_cand, active contract version v_curr
Output: GateStatus in {PASS, REGRESSION_DETECTED, INCONCLUSIVE}

1. h_cand := compute_canonical_ast_hash(F_cand)
2. R := Ledger.get_by_task(task_id)
3. if R is empty: return PASS
4. if enable_ast_filter:
     for each ce in R:
       if ce.ast_hash == h_cand:
         if enforce_contract_version and ce.contract_version != v_curr:
           if not enable_witness_replay: return INCONCLUSIVE (triggered_by="contract_mismatch")
           else: continue
         return REGRESSION_DETECTED (triggered_by="ast_filter", matched_ce_id=ce.ce_id)
5. inconclusive_hit := None
6. if enable_witness_replay:
     for each ce in R:
       exec_res := IsolatedTestRunner.run_test_code(F_cand, ce.witness_test_code)
       if exec_res.timed_out or exec_res.execution_error:
         return INCONCLUSIVE (triggered_by="replay_error", matched_ce_id=ce.ce_id)
       if not exec_res.passed:
         if enforce_contract_version and ce.contract_version != v_curr:
           inconclusive_hit := INCONCLUSIVE (triggered_by="contract_mismatch", matched_ce_id=ce.ce_id)
           continue
         return REGRESSION_DETECTED (triggered_by="witness_replay", matched_ce_id=ce.ce_id)
7. if inconclusive_hit is not None: return inconclusive_hit
8. return PASS
```

4. System Architecture & Core Verification Components

CEGAR-Harness is engineered as a modular Python package (`cegar_harness`) with zero heavyweight external service dependencies, using Python's built-in `sqlite3`, `ast`, and `subprocess` modules alongside the OpenTelemetry SDK (`opentelemetry-sdk`) for distributed tracing.



4.1 Syntactic AST Canonicalization (`_AlphaNormalizer`) vs. Behavioral Witness Replay

To understand how Stage 1 and Stage 2 interact inside `RegressionGate`, consider three candidate patches for `compact_ints` (Task `T26`). Stage 1 parses the candidate Python source and normalizes docstrings, comments, whitespace, and user-defined local identifiers:

Table 3: Concrete comparison of Stage 1 (Canonical AST Hash) and Stage 2 (Witness Replay) across three post-compaction patches.

Candidate Patch Type	Source Code Submitted by Agent	Stage 1: Canonical AST	Stage 2: Witness Replay

Pre-Compaction Failure (Recorded as `CE-T26-01`)	`def compact_ints(items):`
	`  """Strip None values."""`
	`  out = []`
	`  for item in items:`
	`  if item:`
	`  out.append(item)`
	`  return out`
		`h_AST = 9f1c8a...` (Parameters/locals mapped to `_v0, _v1, _v2`; docstring removed)	Fails `test_zero_is_not_dropped`: `[1, 2] != [0, 1, 2]`

Alpha-Renamed Repeat (Categories 1–5)	`def compact_ints(seq):`
	`  # Renamed vars, no docstring`
	`  res = []`
	`  for val in seq:`
	`  if val:`
	`  res.append(val)`
	`  return res`
		MATCH (`9f1c8a...`) Caught in **0.30 ms** without spawning a subprocess.	Skipped (Short-circuited by Stage 1 AST match).

Syntactic Variant Rewrite (Category 6, `T26-c1-synvar`)	`def compact_ints(items):` `  return [x for x in items if x]`	**MISS** (`4b82e1...`) `ListComp` node differs structurally from `For` loop.	**CAUGHT in 18.9 ms**: Replaying `CE-T26-01` raises `AssertionError: [1, 2] != [0, 1, 2]`.

4.2 Isolated Subprocess Witness Replay & Path-Traversal Confinement

`IsolatedTestRunner` (in `cegar_harness/replay.py`) materializes candidate files inside a fresh ephemeral directory created via `tempfile.TemporaryDirectory(prefix="cegar_replay_")`. Before writing any file, it resolves the target path and enforces strict prefix confinement (`str(target).startswith(str(root.resolve()))`), immediately rejecting any `../` path traversal attempt with `SecurityError` (verified in `tests/test_gate_and_replay.py::test_replay_blocks_path_traversal`). Witness functions (`test_*`) execute in a clean child Python interpreter invoked with `python -B`, a sanitized environment (`PYTHONDONTWRITEBYTECODE=1`, isolated `PYTHONPATH`), and a hard wall-clock timeout (default `5.0s`).

4.3 Industrial Integration: MCP Server, Atomic Git Workspace Adapter, and LangGraph

To enable plug-and-play adoption in existing agentic coding stacks without modifying core LLM orchestration code, CEGAR-Harness exposes three production integration interfaces alongside an interactive inspection dashboard:

1. **Model Context Protocol (MCP) JSON-RPC 2.0 Server (`cegar_harness/mcp_server.py`, CLI: `cegar-harness mcp-serve`):** Implements a compliant MCP server over `stdio` (protocol version `2024-11-05`) that can be registered directly in Claude Code (`.mcp.json`) or Cursor. It exposes four tools:
   - `cegar_evaluate_patch`: Evaluates candidate file mappings against active witnesses in SQLite and returns the structured gate verdict before the agent commits edits to disk.
   - `cegar_record_witness`: Computes the canonical AST hash, extracts the exact assertion failure predicate via `IsolatedTestRunner` if not already provided, and records a versioned `CounterexampleRecord`.
   - `cegar_compact_trajectory`: Compresses an array of trajectory turns under any of the 5 strategies while automatically pinning active counterexamples for the task.
   - `cegar_query_ledger`: Queries stored witnesses filtered by `task_id`, `target_file`, or `target_symbol`.

```
// Example MCP JSON-RPC 2.0 Tool Call & Response over stdio (cegar_evaluate_patch)
--> {"jsonrpc": "2.0", "id": 4, "method": "tools/call", "params": {
      "name": "cegar_evaluate_patch",
      "arguments": {
        "task_id": "T26",
        "candidate_files": {"filters.py": "def compact_ints(items):
    return [x for x in items if x]
"},
        "contract_version": "v1"
      }
    }}
<-- {"jsonrpc": "2.0", "id": 4, "result": {
      "content": [{"type": "text", "text": "{"status": "REGRESSION_DETECTED", "matched_ce_id": "CE-T26-01",
"triggered_by": "witness_replay", "reason": "Candidate failed stored witness test
'test_zero_is_not_dropped' (CE-T26-01): ASSERTION_FAILED: test_zero_is_not_dropped: assert [1, 2] == [0, 1,
2]", "latency_ms": 18.91, "ledger_lookup_ms": 0.35, "replay_latency_ms": 18.56}"}],
      "isError": false
    }}
```

2. **Atomic Git Workspace Adapter (`cegar_harness/git_workspace.py`):** Wraps a local Git repository (`GitWorkspaceAdapter`), allowing an agent to apply either a file dictionary (`apply_candidate_with_gate`) or a unified diff via `git apply` (`apply_unified_diff_with_gate`), capture the exact working-tree `git diff`, and evaluate `RegressionGate`. If `status == GateStatus.REGRESSION_DETECTED`, the adapter automatically restores all snapshotted files, deletes any newly created untracked files, and executes `git checkout -- .`, guaranteeing atomic rollback.
3. **LangGraph State-Machine Middleware (`cegar_harness/langgraph_adapter.py`):** Provides `CEGARHarnessMiddleware` and `AgentGraphState` to wrap tool-execution nodes in LangGraph workflows. When a code-edit tool call is intercepted and blocked by `RegressionGate`, the middleware prevents the tool write, appends a structured `[CEGAR_REGRESSION_BLOCKED]` observation containing the falsifying witness predicate to `state.turns`, increments `state.regression_blocks`, and automatically compacts the trajectory whenever token count exceeds `token_budget`.
4. **Interactive Web Inspector UI (`cegar_harness/ui_server.py`, CLI: `cegar-harness ui --port 8080`):** Serves a zero-dependency local web application backed by Python's `ThreadingHTTPServer` with three interactive views: (1) a Side-by-Side Context Compaction Diff Viewer across all 30 tasks, (2) a Live Regression Gate & Witness Replay Sandbox (`POST /api/evaluate`) with editable Python code and ablation toggles, and (3) a Benchmark, 4-Way Ablation & OpenTelemetry Span Explorer.

5. Experimental Methodology & 30-Task Benchmark Suite

To evaluate CEGAR-Harness without confounding harness mechanics with unrepeatable API sampling noise, our experimental protocol separates evaluation into two complementary tracks:

- **Experiment A (Multi-Turn Controlled Replay & 4-Way Component Ablation):** Deterministic, fully reproducible execution across 30 Python code-repair tasks spanning 6 bug categories under a strict post-compaction reference oracle budget of `max_oracle_turns = 2`.
- **Experiment B (Multi-Turn Live LLM Agent Repair Loop):** Connects `LiveLLMClient` to any OpenAI-compatible or local Ollama endpoint across up to `max_turns = 3` post-compaction self-repair turns, feeding back `RegressionGate` diagnostics whenever a candidate patch is blocked. In accordance with strict scientific reporting rules, when no live LLM endpoint is configured in the environment, Experiment B explicitly reports `NOT_EXECUTED_NO_CONFIG` without fabricating or simulating LLM completions.

Table 4: Complete catalog of the 30 executable Python code-repair tasks across 6 categories in CEGAR-Harness.

ID	Category	Target Module & Symbol	Bug Description	Contract	Partial CE
T01	`1_boundary_conditions`	`pagination.py::clamp_limit`	Pagination limit clamping at zero boundary	v1 → v1	Yes (1)
T02	`1_boundary_conditions`	`stats.py::rolling_averages`	Rolling window average off-by-one when len == window	v1 → v1	No
T03	`1_boundary_conditions`	`bsearch.py::search_leftmost`	Leftmost binary search with duplicate elements	v1 → v1	Yes (1)
T04	`1_boundary_conditions`	`batching.py::chunk_items`	Batch chunking on empty input list	v1 → v1	No
T05	`1_boundary_conditions`	`pricing.py::apply_discount`	Inclusive discount threshold boundary	v1 → v1	No
T06	`2_parsers_state_machines`	`csv_parser.py::split_csv_row`	CSV splitter preserving commas inside quotes	v1 → v1	No
T07	`2_parsers_state_machines`	`brackets.py::is_balanced`	Bracket balancer unclosed stack check	v1 → v1	No
T08	`2_parsers_state_machines`	`ini_parser.py::parse_kv_lines`	Key-value INI parser preserving '=' in values	v1 → v1	Yes (1)
T09	`2_parsers_state_machines`	`lexer.py::tokenize_args`	Shell argument lexer with backslash-escaped spaces	v1 → v1	No
T10	`2_parsers_state_machines`	`protocol.py::next_state`	Protocol state machine RESET transition from ERROR state	v1 → v1	Yes (1)
T11	`3_exception_handling`	`converters.py::safe_int`	Safe integer conversion when input is None	v1 → v1	No
T12	`3_exception_handling`	`validators.py::parse_pct`	Percentage parser rejecting >100% or <0% with ValueError	v1 → v1	No
T13	`3_exception_handling`	`lookup.py::deep_get`	Nested dictionary lookup raising KeyError when intermediate value is None	v1 → v1	Yes (1)
T14	`3_exception_handling`	`runner_utils.py::run_with_cleanup`	Guaranteed cleanup callback execution via try/finally	v1 → v1	No
T15	`3_exception_handling`	`weights.py::normalize_weights`	NaN and Inf rejection in positive weight normalization	v1 → v1	No
T16	`4_multi_file_changes`	`session_guard.py::is_token_valid`	Cross-module token expiry unit alignment (ms vs seconds)	v1 → v1	No
T17	`4_multi_file_changes`	`serializer.py::serialize_user`	Cross-module email normalization in schema and serializer	v1 → v1	No

ID	Category	Target Module & Symbol	Bug Description	Contract	Partial CE
T18	4_multi_file_changes	api_gateway.py::handle_request	Rate limiter tuple return unpacking in API gateway	v1 → v1	No
T19	4_multi_file_changes	invoice.py::compute_invoice_total	Tax-exempt line items exclusion across invoice and tax_rules	v1 → v1	No
T20	4_multi_file_changes	path_finder.py::shortest_hop_count	Cycle-safe BFS shortest path across graph_store and path_finder	v1 → v1	Yes (2)
T21	5_feature_regressions	slug.py::slugify	Slug separator contract evolution (v1 underscore -> v2 hyphen)	v1 → v2	No
T22	5_feature_regressions	config_loader.py::get_config_value	Config lookup missing-key policy evolution (v1 None -> v2 KeyError)	v1 → v2	No
T23	5_feature_regressions	shipping.py::shipping_fee	Heavy freight shipping fee policy evolution (v1 free >=50kg -> v2 surcharge)	v1 → v2	No
T24	5_feature_regressions	redactor.py::redact_payload	PII redaction token evolution (v1 '***' -> v2 '[REDACTED]')	v1 → v2	No
T25	5_feature_regressions	money.py::format_usd	Negative currency formatting evolution (v1 '-$12.50' -> v2 '($12.50)')	v1 → v2	No
T26	6_syntactic_variants	scores.py::keep_valid_scores	Zero score dropped by truthiness filter (for-loop vs list comprehension)	v1 → v1	No
T27	6_syntactic_variants	collections_ext.py::dedup_ordered	Order-destroying deduplication (sorted(set) vs list.sort())	v1 → v1	No
T28	6_syntactic_variants	options.py::resolve_timeout	Zero timeout overridden by truthy fallback (if-else vs boolean `or`)	v1 → v1	Yes (1)
T29	6_syntactic_variants	path_utils.py::normalize_relpath	Parent directory '..' resolution in path normalizer (comprehension vs loop skip)	v1 → v1	Yes (1)
T30	6_syntactic_variants	metrics.py::median_value	Even-length median calculation (upper-middle index vs float average)	v1 → v1	Yes (2)

6. Empirical Results & Statistical Analysis

6.1 RQ1: Multi-Turn Context Compaction Strategies Comparison (30 Tasks)

In Experiment A, each task begins with a 6-turn pre-compaction trajectory where the initial defective patch fails its witness test at Step 2 (`CE-Txx-01`), followed by two repository and static-analysis exploration turns (mean pre-compaction length: `666.8 tokens`). Compaction is then triggered, and the agent attempts post-compaction repair under a strict budget of `max_oracle_turns = 2` full reference test-suite executions. Nine tasks (`T01`, `T03`, `T08`, `T10`, `T13`, `T20`, `T28`, `T29`, `T30`) include **Partial Witness Coverage** intermediate candidates that satisfy the initial witness `CE-01` (so the first-round gate returns `PASS`) but fail a secondary boundary or invariant in the reference suite, requiring a second CEGAR refinement round. Two multi-constraint tasks (`T20` and `T30`) feature two chained partial-coverage faults.

Table 5: Empirical comparison of the 5 context compaction strategies across all 30 tasks (with 95% Wilson confidence intervals).

Compaction Strategy	Mean Tokens (Pre → Post)	Comp. (%)	Witness Retained	Gate Blocked	Unblocked Reg. (%) [95% CI]	2nd-Round CEs	Turn-1 Resolved (%)	Final Resolved (%) [95% CI]	Mean Oracle Turns
FULL_HISTORY	666.8 → 666.8	0.00%	100.0%	0	16.67% [7.3, 33.6]	0	63.33% (19/30)	**86.67%** (26/30) [70.3, 94.7]	1.37
OBSERVATION_MASKING	666.8 → 404.9	39.33%	0.0%	0	100.00% [88.7, 100.0]	0	0.00% (0/30)	**70.00%** (21/30) [52.1, 83.3]	2.00
LOSSY_SUMMARY	666.8 → 312.7	53.04%	0.0%	0	100.00% [88.7, 100.0]	0	0.00% (0/30)	**56.67%** (17/30) [39.2, 72.6]	2.00
CEGAR_LEDGER_ONLY	666.8 → 344.6	48.26%	100.0%	0	16.67% [7.3, 33.6]	10	63.33% (19/30)	**86.67%** (26/30) [70.3, 94.7]	1.37
CEGAR_HARNESS_FULL	666.8 → 344.6	48.26%	100.0%	5	0.00% [0.0, 11.3]	11	70.00% (21/30)	**93.33%** (28/30) [78.7, 98.2]	1.30

Analysis of RQ1 Findings:

1. **Collapse of Turn-1 Resolution under Standard Compaction:** Both `OBSERVATION_MASKING` (`39.33%` compression) and `LOSSY_SUMMARY` (`53.04%` compression) lose `100%` of pre-compaction witness predicates (`0.0%` retention). Consequently, both suffer a **100.00% (30/30)** Turn-1 unblocked regression rate and **0.00% Turn-1 resolution**. Because Turn 1 is wasted re-running the known regression against the full test suite, any task with a subsequent partial-witness fault (`9 tasks` for masking; `13 tasks` including

multi-file distractors for lossy summarization) exhausts the 2-turn oracle budget, yielding final resolution rates of only **70.00% (21/30)** (95% CI: [52.1%, 83.3%]) and **56.67% (17/30)** (95% CI: [39.2%, 72.6%]), respectively.

2. **Prompt-Only Ledger Pinning vs. Pre-Execution Gating:** CEGAR_LEDGER_ONLY compresses context by **48.26%** (666.8 → 344.6 tokens) while matching the uncompacted FULL_HISTORY baseline on every metric: **63.33% (19/30)** Turn-1 resolution and **86.67% (26/30)** final resolution. However, both suffer a **16.67% (5/30)** unblocked regression rate on Category 6 (6_syntactic_variants), where prompt text alone fails to prevent syntactically distinct rewrites of the refuted bug. On tasks T28 and T29, wasting Oracle Turn 1 on the unblocked syntactic variant causes the subsequent partial-witness candidate to exhaust Turn 2.
3. **Gain from Pre-Execution Witness Replay (CEGAR_HARNESS_FULL):** By intercepting all 5 Category 6 syntactic variants in the pre-execution RegressionGate (Blocked = 5, 0.00% unblocked regressions), CEGAR_HARNESS_FULL preserves both oracle turns for genuine new candidates. Tasks T28 and T29 test their partial-coverage patch on Turn 1, synthesize a secondary counterexample (CE-02), and converge on the valid fix on Turn 2. Only the 2 tasks with two chained partial-coverage faults (T20 and T30) exhaust the 2-turn budget, resulting in a **70.00% (21/30)** Turn-1 resolution rate, a **93.33% (28/30)** final resolution rate (95% CI: [78.7%, 98.2%]), and the lowest mean oracle turns to convergence (**1.30 turns**).

6.2 RQ2 & RQ3: Four-Way Regression Gate Ablation Study (60 Candidate Evaluations)

To isolate the causal contribution of each gate mechanism, we evaluate four ablation configurations of RegressionGate across all 30 tasks (evaluating both the post-compaction regression candidate c1 and the valid solution candidate c2, for 60 total gate evaluations per configuration):

Table 6: Four-way ablation study of RegressionGate across 30 tasks (60 candidate evaluations per configuration).

Ablation Config	AST	Replay	Ver.	Precision	Recall (Caught)	F1-Score	Cat. 6 Syn. Recall	False Block Rate	Inconc.	Resolved	Med / p95 (ms)	Ledger / Replay (ms)
full_cegar_harness	ON	ON	ON	100.00%	100.00% (30/30)	**100.00%**	100.0% (5/5)	0.00% (0/30)	5	100.00% (30/30)	18.71 / 21.21	0.36 / 11.15
ast_filter_only	ON	OFF	ON	100.00%	83.33% (25/30)	**90.91%**	0.0% (0/5)	0.00% (0/30)	0	83.33% (25/30)	0.30 / 0.52	0.31 / 0.00
witness_replay_only	OFF	ON	ON	100.00%	100.00% (30/30)	**100.00%**	100.0% (5/5)	0.00% (0/30)	5	100.00% (30/30)	19.93 / 22.44	0.37 / 19.73
unversioned_ledger	ON	ON	OFF	85.71%	100.00% (30/30)	**92.31%**	100.0% (5/5)	16.67% (5/30)	0	83.33% (25/30)	18.74 / 22.28	0.34 / 11.36

Analysis of RQ2 (AST Filter vs. Witness Replay) & RQ3 (Contract Versioning):

- **Failure of AST Hashing on Syntactic Variants (ast_filter_only):** Disabling behavioral Witness Replay reduces median gate latency to 0.30 ms (0.62 ms p95), catching all 25 alpha-renamed regressions in Categories 1–5. However, its recall on Category 6 (6_syntactic_variants) drops to **0.0% (0/5)**, reducing overall recall to **83.33% (25/30)** and F1 to **90.91%**.
- **Latency Optimization from Combining AST Pre-Filter with Witness Replay:** While witness_replay_only achieves 100.00% precision and recall by running a subprocess on every candidate (mean replay overhead 19.61 ms), enabling the AST pre-filter in full_cegar_harness short-circuits exact/alpha-renamed repeats in 0.36 ms without spawning a Python subprocess, cutting mean subprocess replay overhead by **40.9%** (from 19.61 ms to 11.59 ms) while maintaining 100.00% F1.
- **False Block Surge without Contract Versioning (unversioned_ledger):** When enforce_contract_version=False, all 5 evolved-contract tasks in Category 5 (T21–T25, where requirements transition from v1 → v2) fail their stale v1 witness during replay and are **falsely blocked** (**16.67% False Block Rate**, 5/30 valid solutions rejected; precision drops to **85.71%**). Enabling contract versioning transitions these 5 evaluations to GateStatus.INCONCLUSIVE, restoring **0.00% False Block Rate** and **100.00% Precision**.

6.3 Per-Category Regression & Multi-Turn Convergence Breakdown

Table 7: Breakdown of unblocked regressions, Turn-1 vs. Final resolution, secondary witnesses, and inconclusive gates across the 6 bug categories.

Bug Category	Tasks	Obs. Masking Unblk. Reg.	Obs. Masking Resolved	Ledger-Only Unblk. Reg.	Ledger-Only Resolved	CEGAR Full Unblk. Reg.	CEGAR Full Turn-1 Res.	CEGAR Full Final Res.	2nd-Round CEs	Inconc. Gates
1_boundary_conditions	5	5/5	3/5	0/5	5/5	0/5	3/5	**5/5**	2	0
2_parsers_state_machines	5	5/5	3/5	0/5	5/5	0/5	3/5	**5/5**	2	0
3_exception_handling	5	5/5	4/5	0/5	5/5	0/5	4/5	**5/5**	1	0
4_multi_file_changes	5	5/5	4/5	0/5	4/5	0/5	4/5	**4/5**	2	0
5_feature_regressions	5	5/5	5/5	0/5	5/5	0/5	5/5	**5/5**	0	5
6_syntactic_variants	5	5/5	2/5	5/5	2/5	0/5	2/5	**4/5**	4	0

6.4 Complete Per-Task Empirical Evaluation Matrix (Tasks T01–T30)

Table 8: Per-task compaction measurements and multi-turn resolution outcomes across all 30 tasks (T = oracle turns used out of 2).

Task	Category	Tokens (Pre → Post, Saved%)	Lossy Summary	Obs. Masking	CEGAR Ledger-Only	Gate Blocked	2nd CEs	Inconc.	CEGAR Full
T01	1_boundary_conditions	639 → 337 (47.3%)	FAIL (T=2)	FAIL (T=2)	PASS (T=2)	0	1	0	**PASS** (T=2)
T02	1_boundary_conditions	686 → 349 (49.1%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T03	1_boundary_conditions	695 → 350 (49.6%)	FAIL (T=2)	FAIL (T=2)	PASS (T=2)	0	1	0	**PASS** (T=2)
T04	1_boundary_conditions	656 → 338 (48.5%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T05	1_boundary_conditions	667 → 351 (47.4%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T06	2_parsers_state_machines	656 → 352 (46.3%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T07	2_parsers_state_machines	692 → 334 (51.7%)	FAIL (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T08	2_parsers_state_machines	708 → 360 (49.1%)	FAIL (T=2)	FAIL (T=2)	PASS (T=2)	0	1	0	**PASS** (T=2)
T09	2_parsers_state_machines	664 → 359 (45.9%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T10	2_parsers_state_machines	725 → 347 (52.1%)	FAIL (T=2)	FAIL (T=2)	PASS (T=2)	0	1	0	**PASS** (T=2)
T11	3_exception_handling	648 → 350 (46.0%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T12	3_exception_handling	646 → 338 (47.7%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T13	3_exception_handling	642 → 335 (47.8%)	FAIL (T=2)	FAIL (T=2)	PASS (T=2)	0	1	0	**PASS** (T=2)
T14	3_exception_handling	626 → 332 (47.0%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T15	3_exception_handling	662 → 331 (50.0%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T16	4_multi_file_changes	676 → 337 (50.1%)	FAIL (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T17	4_multi_file_changes	701 → 347 (50.5%)	FAIL (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T18	4_multi_file_changes	687 → 350 (49.0%)	FAIL (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T19	4_multi_file_changes	684 → 337 (50.7%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	0	**PASS** (T=1)
T20	4_multi_file_changes	736 → 344 (53.3%)	FAIL (T=2)	FAIL (T=2)	FAIL (T=2)	0	2	0	**FAIL** (T=2)
T21	5_feature_regressions	634 → 328 (48.3%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	1	**PASS** (T=1)
T22	5_feature_regressions	649 → 341 (47.5%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	1	**PASS** (T=1)
T23	5_feature_regressions	640 → 341 (46.7%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	1	**PASS** (T=1)
T24	5_feature_regressions	678 → 369 (45.6%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	1	**PASS** (T=1)
T25	5_feature_regressions	634 → 339 (46.5%)	PASS (T=2)	PASS (T=2)	PASS (T=1)	0	0	1	**PASS** (T=1)
T26	6_syntactic_variants	659 → 349 (47.0%)	PASS (T=2)	PASS (T=2)	PASS (T=2)	1	0	0	**PASS** (T=1)
T27	6_syntactic_variants	645 → 352 (45.4%)	PASS (T=2)	PASS (T=2)	PASS (T=2)	1	0	0	**PASS** (T=1)
T28	6_syntactic_variants	642 → 339 (47.2%)	FAIL (T=2)	FAIL (T=2)	FAIL (T=2)	1	1	0	**PASS** (T=2)
T29	6_syntactic_variants	674 → 351 (47.9%)	FAIL (T=2)	FAIL (T=2)	FAIL (T=2)	1	1	0	**PASS** (T=2)
T30	6_syntactic_variants	653 → 351 (46.2%)	FAIL (T=2)	FAIL (T=2)	FAIL (T=2)	1	2	0	**FAIL** (T=2)

6.5 Statistical Confidence Intervals & Latency Decomposition

Because binomial proportions on sample size $n = 30$ exhibit asymmetric sampling distributions near boundary rates (0% and 100%), normal Wald intervals degenerate to zero width at $\hat{p} = 0$ or $\hat{p} = 1$. In `cegar_harness/benchmark.py` (`wilson_score_interval`), all reported 95% confidence intervals ($z = 1.95996$) are computed via the **Wilson score interval**:

$$CI_{95\%}(k, n) = [\ (\hat{p} + z^2/2n \pm z\ \sqrt{(\ \hat{p}(1 - \hat{p})/n + z^2/4n^2\ )}\ )\ /\ (1 + z^2/n)\ ]$$

For `CEGAR_HARNESS_FULL`, the unblocked regression rate of `0/30 (0.00%)` has a 95% Wilson upper bound of `11.4%`, compared to `30/30 (100.00%, CI: [88.6%, 100.0%])` for `OBSERVATION_MASKING` and `LOSSY_SUMMARY`—a non-overlapping separation exceeding 77 percentage points even at the 95% confidence boundaries.

7. Deep-Dive Case Studies: Partial Witness Coverage & Contract Evolution

7.1 Case Study 1: Multi-Round Partial Witness Coverage on Task T20 and Task T30

Why does `CEGAR_HARNESS_FULL` achieve `93.33% (28/30)` rather than `100.00%` within the 2-turn post-compaction oracle budget? Examining Tasks `T20` and `T30` in Table 8 illustrates a fundamental property of counterexample-guided verification: *a counterexample witness is an under-approximation of the full specification.*

- **Task `T20` (`path_finder.py::shortest_hop_count`):** The pre-compaction patch omits a `visited` set during Breadth-First Search (BFS), entering an infinite loop on cyclic graphs when the destination is unreachable. Witness `CE-T20-01` tests `shortest_hop_count({'A': ['B'], 'B': ['A']}, 'A', 'Z') == -1`. After compaction:
  1. On Post-Compaction Turn 1, candidate `T20-c1b-partial1` introduces a `visited` set, which **passes** `CE-T20-01` (`RegressionGate` returns `PASS`). However, when executed against the full reference oracle, it fails on the self-loop edge case `shortest_hop_count(adj, 'A', 'A') == 0` (returning `2` via neighbor cycle `A → B → A`), consuming Oracle Turn 1 and synthesizing secondary witness `CE-T20-02`.
  2. On Post-Compaction Turn 2, candidate `T20-c1c-partial2` adds the `if src == dst: return 0` guard (passing `CE-T20-01`), but initializes the queue with distance `1` instead of `0`, failing `shortest_hop_count(adj, 'A', 'D') == 2` on Oracle Turn 2 and synthesizing `CE-T20-03`.
  3. Because the strict 2-turn oracle budget (`max_oracle_turns = 2`) is reached before the third candidate (`T20-c2-fix`), Task `T20` is recorded as unresolved at $T_{max} = 2$ (converging on Turn 3 if $T_{max} \geq 3$).
- **Task `T30` (`metrics.py::median_value`):** Witness `CE-T30-01` tests even-length median averaging on an already-sorted input `[1.0, 2.0, 4.0, 10.0] == 3.0`. Post-compaction candidate `T30-c1-synvar` (an in-place sort indexing single upper-middle element) is immediately blocked in `18.9 ms` by `RegressionGate` (`REGRESSION_DETECTED`) without consuming an oracle turn. However, the subsequent partial-coverage candidates `T30-c1b-partial1` (which averages the middle two elements without sorting first, passing the sorted witness `[1.0, 2.0, 4.0, 10.0]` but failing unsorted `[10.0, 2.0, 5.0]`) and `T30-c1c-partial2` (which sorts and averages even lengths, passing the witness, but applies even-averaging unconditionally to odd-length lists) consume Oracle Turns 1 and 2.

In contrast, on tasks with a single partial-coverage refinement step such as `T28` (`options.py::resolve_timeout`) and `T29` (`path_utils.py::normalize_relpath`), blocking the initial syntactic-variant regression in `RegressionGate` saves Turn 1 for the partial candidate (synthesizing `CE-02`) and allows the agent to resolve the task on Turn 2. Under `CEGAR_LEDGER_ONLY`, where no pre-execution gate runs, Turn 1 is wasted on the syntactic variant and Turn 2 is consumed by the partial candidate, causing both `T28` and `T29` to fail within the 2-turn budget.

7.2 Case Study 2: Contract Evolution (`v1 → v2`) on Category 5 (Tasks T21–T25)

Category 5 evaluates what happens when a user or specification updates the expected behavior of a function mid-session:

- **Task `T21` (`discounts.py::apply_discount`):** Under contract `v1`, a discount exceeding price clamps to `0.0`; witness `CE-T21-01` asserts `apply_discount(50.0, 60.0) == 0.0`. After compaction, contract `v2` specifies that `discount > price` must raise `ValueError("Discount exceeds price")`.
- When the valid `v2` patch is evaluated by `unversioned_ledger`, replaying `CE-T21-01` raises `ValueError`, causing the gate to return `REGRESSION_DETECTED` and reject the valid solution.
- When evaluated by `full_cegar_harness`, the gate detects `ce.contract_version ("v1") != current_contract_version ("v2")` and returns `GateStatus.INCONCLUSIVE` (`triggered_by="contract_mismatch"`), allowing the candidate to proceed to the `v2` reference oracle where it passes.

7.3 Multi-Turn Live Agent Self-Repair Protocol (Experiment B)

In addition to controlled replay (Experiment A), `cegar_harness/benchmark.py` (`run_experiment_b_live_llm`) implements an autonomous multi-turn repair loop (`max_turns = 3` post-compaction) over four strategies (`LOSSY_SUMMARY`, `OBSERVATION_MASKING`, `CEGAR_LEDGER_ONLY`, and `CEGAR_HARNESS_FULL`):

```
For each task T and strategy S in {LOSSY_SUMMARY, OBSERVATION_MASKING, CEGAR_LEDGER_ONLY,
CEGAR_HARNESS_FULL}:
```

```
  1. Seed pre-compaction witness CE-01 into Ledger L and compact trajectory T_pre using strategy S.
  2. For turn = 1 .. max_turns (3):
       a. Prompt LLM with compacted context + repair_feedback_history to synthesize candidate Python file
F_cand.
       b. Evaluate GateDecision := RegressionGate.evaluate(T.task_id, F_cand,
T.contract_version_post_compaction).
       c. If S == CEGAR_HARNESS_FULL and GateDecision.status == REGRESSION_DETECTED:
            // Pre-execution self-repair feedback without running full reference suite
            Append "[CEGAR RegressionGate BLOCKED Turn t] Matched CE: {matched_ce_id} ({reason})"
            to repair_feedback_history and continue to turn + 1.
       d. Execute full reference test suite Phi_ref on F_cand:
            If passed: record (resolved=True, turns_used=turn) and break.
            Else: extract new failure predicate psi_fail; if S in {CEGAR_LEDGER_ONLY, CEGAR_HARNESS_FULL},
                  record secondary witness CE-0(turn+1) in Ledger L and append feedback for turn + 1.
```

This multi-turn repair loop is verified end-to-end by an integration test in `tests/test_benchmark_integrity.py::test_experiment_b_multi_turn_self_repair_loop`, which confirms that when Turn 1 proposes a syntactic-variant regression (`[x for x in items if x]` on Task `T26`), `CEGAR_HARNESS_FULL` blocks the patch in the gate, injects the exact `CE-T26-01` diagnostic into the Turn 2 prompt, and converges on the valid patch (`[x for x in items if x is not None]`) on Turn 2.

8. Discussion, Limitations & Threats to Validity

1. **Construct Validity & Controlled Replay vs. Live LLM Sampling:** Experiment A uses deterministic candidate queues conditioned on whether the witness predicate survives compaction. This design guarantees 100% deterministic reproducibility in CI and isolates the exact causal effect of AST hashing, witness replay, and contract versioning. However, live LLMs exhibit stochastic variation across model families and temperatures; Experiment B (`cegar-harness live-eval --api-base ... --model ...`) provides the multi-turn live evaluation loop to measure model-specific repair trajectories.
2. **Witness Brittleness & Over-Specification:** Witness replay assumes that each stored `witness_test_code` tests a behavioral contract rather than private implementation internals. If a witness asserts internal helper names that are legitimately refactored under the same contract version, the gate could trigger a false block unless the contract version is incremented or the witness is marked `OBSOLETE`.
3. **Subprocess Replay Overhead at Scale:** Spawning an isolated Python interpreter per witness incurs a median latency of `18.89 ms` (`21.72 ms` p95). While negligible for 1–5 active witnesses per task (especially compared to 2,000–10,000 ms LLM generation calls), sessions accumulating dozens of witnesses across a large repository should index witnesses by `(target_file, target_symbol)` and batch all matching witness assertions into a single subprocess invocation.

9. Conclusion & Reproducibility

We have presented **CEGAR-Harness**, an open-source middleware and benchmark suite that addresses Post-Compaction Regression Loops in coding agents through versioned counterexample ledgers, counterexample-pinned context compaction, and a three-state pre-execution Regression Gate. Across 30 multi-turn Python repair tasks, CEGAR-Harness achieves a **48.26%** token compression ratio, reduces unblocked known regressions from **100.00%** to **0.00%**, prevents **100% (5/5)** of false blocks on evolved contracts via its `INCONCLUSIVE` state, and reaches a **93.33%** two-turn task resolution rate. All source code, the 30-task benchmark, the MCP server, the Git workspace adapter, the LangGraph middleware, and the Web Inspector UI are publicly available under the MIT License at `https://github.com/iyedM/cegar-harness` and can be reproduced in under 20 seconds via `bash scripts/run_benchmark.sh`.

References

1. J. Cai et al., "Compact-Bench: Evaluating Context Compaction Damage in Autonomous Coding Agents," *arXiv preprint arXiv:2610.31710*, Oct. 2026. Code: `https://github.com/cai-jianfeng/compact-bench`
2. M. Kang et al., "ACON: Optimizing Context Compression for Long-horizon LLM Agents," *arXiv preprint arXiv:2510.00615*, Oct. 2025.
3. JetBrains Research, "The Complexity Trap: Simple Observation Masking vs. LLM Summarization in Software Engineering Agents," *JetBrains AI Research Publications*, 2025.
4. Microsoft Research, "CompactionBench and APEC: Asynchronous Progressive Context Compaction for Long-Horizon Coding Sessions," *Microsoft Research Technical Report*, Oct. 2026.
5. Y. Yang et al., "ExVerus: Counterexample-Guided Proof Repair with Large Language Models," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, PMLR vol. 306, 2026.
6. E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-Guided Abstraction Refinement for Symbolic Model Checking," *Journal of the ACM (JACM)*, vol. 50, no. 5, pp. 752–794, 2003.
7. C. E. Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," in *International Conference on Learning Representations (ICLR)*, 2024.